CLASS 21+22 HOMEWORK

Authentication & Build Process

7 – 8 hours	100 points total	■ AI tools encouraged

■ What This Covers

Two projects: build a complete login system from scratch, then prepare your mini project for production (env variables, build, lazy loading, README). After this class there is ONE more session before your Capstone begins.

PROJECT 1 · Full Login / Signup System · 60 pts

■ What You Are Building

Build a complete auth flow using React Context, React Hook Form, and the free reqres.in mock API. No real backend needed. The patterns here are identical to how production auth systems work.

STATE & CONTEXT

user — The logged-in user object: { email, token }. null when logged out.

login(user) — Saves user to Context state AND localStorage. Called after successful API login.

logout() — Clears user from Context state AND localStorage. Redirect after calling.

useAuth() — Custom hook — any component calls this to read user, login, logout.

REQUIREMENTS

- Create **src/context/AuthContext.jsx** — use **function** declarations, not arrow functions
- AuthProvider wraps <App /> in main.jsx
- Login page at **/login** — email + password, built with React Hook Form
- Call **https://reqres.in/api/login** on submit. On success: login(user) → navigate('/dashboard')
- On failed login: show error using **setError('root', { message: '...' })**
- Signup page at **/signup** — email, password, confirm password
- Confirm password uses **validate:** option in register() to compare with watch('password')
- Token + email saved in localStorage so session survives a browser refresh
- Protected Dashboard at **/dashboard** — auto-redirects to /login if not logged in
- Dashboard shows: "Welcome, {user.email}!" and the token value
- Logout clears auth state + localStorage and navigates to /
- Navbar shows "Log In" when logged out and "Log Out" button when logged in

REQRES.IN TEST CREDENTIALS

```
// Use these to test your login form during development Email: eve.holt@reqres.in Password:
cityslicka // Login endpoint POST https://reqres.in/api/login // Body: { "email": "...",
"password": "..." } // Returns: { "token": "QpwL5tpe83ilfN2..." } // Signup endpoint POST
https://reqres.in/api/register // Same credentials work here too
```

→ *Full code: CODE-SAMPLES.md — Sample 3 | Shown on Slide 7*

CONFIRM PASSWORD — VALIDATION HINT

```
// In SignupPage — register confirmPassword with a validate function: const { watch } = useForm()
{...register('confirmPassword', { required: 'Please confirm your password', validate: (value) =>
value === watch('password') || 'Passwords do not match', })}
```

→ *Full code: CODE-SAMPLES.md — Sample 4*

> **■ AI Tip**
> "Generate a complete AuthContext.jsx using function declarations. Include: user state initialized from localStorage, login(userData) that saves to localStorage, logout() that clears localStorage, and a useAuth() custom hook."

> **■ AI Tip**
> "Write a ProtectedRoute component for React Router v6. It should use useAuth() to check if user is logged in. If not, redirect to /login with replace=true."

BONUS

- ★ "Remember Me" checkbox — only persist to localStorage if the box is checked
- ★ Loading spinner while the login API call is in progress

PROJECT 2 · Environment Variables & Build Prep · 40 pts

■ What You Are Doing
Clean up your mini project so it is ready to deploy next class. Move secrets out of code, verify the build succeeds, add lazy loading, and write a proper README.

REQUIREMENTS

- Move ALL API keys out of component files into **.env**
- Use **VITE_** prefix on every variable: VITE_TMDB_KEY, VITE_API_BASE, etc.
- Read them in code with **import.meta.env.VITE_YOUR_KEY**
- Create **.env.example** with the same variable names but placeholder values
- Open **.gitignore** — confirm **.env** is listed (Vite adds it by default)
- Run **npm run build** — it must complete with no errors
- Run **npm run preview** — the production build must load correctly
- Add **lazy()** + **Suspense** to at least 3 page imports in App.jsx
- Search codebase for **console.log** — remove them all
- Update **README.md**: description, features, setup steps, env variables needed

SETUP CHEATSHEET

```
# Step 1 — Create .env in your project root: VITE_TMDB_KEY=your_actual_key_here
VITE_API_BASE=https://api.themoviedb.org/3 # Step 2 — Create .env.example (safe to commit):
VITE_TMDB_KEY=get_key_from_themoviedb.org VITE_API_BASE=https://api.themoviedb.org/3 # Step 3 — Use
in React code: const KEY = import.meta.env.VITE_TMDB_KEY const BASE = import.meta.env.VITE_API_BASE
# Step 4 — Restart dev server after any .env change!
```

→ *Full code: CODE-SAMPLES.md — Sample 9 | Shown on Slide 12*

LAZY LOADING CHEATSHEET

```
// Change regular imports to lazy imports in App.jsx: import { lazy, Suspense } from 'react' //
Before: import HomePage from './pages/HomePage' // After: const HomePage = lazy(() =>
import('./pages/HomePage')) // Wrap Routes in Suspense: <Suspense fallback={<p className='p-8
text-center'>Loading...</p>}> <Routes> <Route path='/' element={<HomePage />} /> ... </Routes>
</Suspense>
```

→ *Full code: CODE-SAMPLES.md — Sample 11 | Shown on Slide 14*

■ AI Tip
"Convert my App.jsx to use React.lazy() for all page imports and wrap Routes in Suspense. Use function declarations throughout. Here is my current App.jsx: [paste]"

Grading Rubric

Area	What We Look For	Pts
Auth Context	function declarations · user state · login/logout · localStorage restore on load	20
Login Page	React Hook Form · API call · token stored · navigate on success · setError root	20
Signup Page	confirm password validate() · API call · error handling	10
Protected Route	Navigate replace · redirects to /login · works on browser refresh	10
Env Variables	All keys in .env · VITE_ prefix · .env.example exists · .gitignore confirmed	15
Build Prep	npm run build succeeds · lazy loading 3+ routes · no console.logs	15
README	Description · setup steps · env variable docs · features list	10

After this class your app is auth-protected, secret-safe, and production-ready. One more session before the Capstone. ■